

[Types and Typeclasses](#)[Table of contents](#)[Recursion](#)

Syntax in Functions

Pattern matching

This chapter will cover some of Haskell's cool syntactic constructs and we'll start with pattern matching. Pattern matching consists of specifying patterns to which some data should conform and then checking to see if it does and deconstructing the data according to those patterns.

When defining functions, you can define separate function bodies for different patterns. This leads to really neat code that's simple and readable. You can pattern match on any data type — numbers, characters, lists, tuples, etc. Let's make a really trivial function that checks if the number we supplied to it is a seven or not.



```
lucky :: (Integral a) => a -> String
lucky 7 = "LUCKY NUMBER SEVEN!"
lucky x = "Sorry, you're out of luck, pal!"
```

When you call `lucky`, the patterns will be checked from top to bottom and when it conforms to a pattern, the corresponding function body will be used. The only way a number can conform to the first pattern here is if it is 7. If it's not, it falls through to the second pattern, which matches anything and binds it to `x`. This function could have also been implemented by using an if statement. But what if we wanted a function that says the numbers from 1 to 5 and says "**Not between 1 and 5**" for any other number? Without pattern matching, we'd have to make a pretty convoluted if then else tree. However, with it:

```
sayMe :: (Integral a) => a -> String
sayMe 1 = "One!"
sayMe 2 = "Two!"
sayMe 3 = "Three!"
sayMe 4 = "Four!"
sayMe 5 = "Five!"
sayMe x = "Not between 1 and 5"
```

Note that if we moved the last pattern (the catch-all one) to the top, it would always say "**Not between 1 and 5**", because it would catch all the numbers and they wouldn't have a chance to fall through and

be checked for any other patterns.

Remember the factorial function we implemented previously? We defined the factorial of a number `n` as `product [1..n]`. We can also define a factorial function *recursively*, the way it is usually defined in mathematics. We start by saying that the factorial of 0 is 1. Then we state that the factorial of any positive integer is that integer multiplied by the factorial of its predecessor. Here's how that looks like translated in Haskell terms.

```
factorial :: (Integral a) => a -> a
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

This is the first time we've defined a function recursively. Recursion is important in Haskell and we'll take a closer look at it later. But in a nutshell, this is what happens if we try to get the factorial of, say, 3. It tries to compute `3 * factorial 2`. The factorial of 2 is `2 * factorial 1`, so for now we have `3 * (2 * factorial 1)`. `factorial 1` is `1 * factorial 0`, so we have `3 * (2 * (1 * factorial 0))`. Now here comes the trick — we've defined the factorial of 0 to be just 1 and because it encounters that pattern before the catch-all one, it just returns 1. So the final result is equivalent to `3 * (2 * (1 * 1))`. Had we written the second pattern on top of the first one, it would catch all numbers, including 0 and our calculation would never terminate. That's why order is important when specifying patterns and it's always best to specify the most specific ones first and then the more general ones later.

Pattern matching can also fail. If we define a function like this:

```
charName :: Char -> String
charName 'a' = "Albert"
charName 'b' = "Broseph"
charName 'c' = "Cecil"
```

and then try to call it with an input that we didn't expect, this is what happens:

```
ghci> charName 'a'
"Albert"
ghci> charName 'b'
"Broseph"
ghci> charName 'h'
*** Exception: tut.hs:(53,0)-(55,21): Non-
exhaustive patterns in function charName
```

It complains that we have non-exhaustive patterns, and rightfully so. When making patterns, we should always include a catch-all pattern so that our program doesn't crash if we get some unexpected input.

Pattern matching can also be used on tuples. What if we wanted to make a function that takes two vectors in a 2D space (that are in the form of pairs) and adds them together? To add together two vectors, we add their x components separately and then their y components separately. Here's how we would have done it if we didn't know about pattern matching:

```
addVectors :: (Num a) => (a, a) -> (a, a) -> (a, a)
addVectors a b = (fst a + fst b, snd a + snd b)
```

Well, that works, but there's a better way to do it. Let's modify the function so that it uses pattern matching.

```
addVectors :: (Num a) => (a, a) -> (a, a) -> (a, a)
addVectors (x1, y1) (x2, y2) = (x1 + x2, y1 + y2)
```

There we go! Much better. Note that this is already a catch-all pattern. The type of **addVectors** (in both cases) is **addVectors :: (Num a) => (a, a) -> (a, a) -> (a, a)**, so we are guaranteed to get two pairs as parameters.

fst and **snd** extract the components of pairs. But what about triples? Well, there are no provided functions that do that but we can make our own.

```
first :: (a, b, c) -> a
first (x, _, _) = x

second :: (a, b, c) -> b
second (_, y, _) = y

third :: (a, b, c) -> c
third (_, _, z) = z
```

The **_** means the same thing as it does in list comprehensions. It means that we really don't care what that part is, so we just write a **_**.

Which reminds me, you can also pattern match in list comprehensions. Check this out:

```
ghci> let xs = [(1,3), (4,3), (2,4), (5,3), (5,6), (3,1)]
ghci> [a+b | (a,b) <- xs]
[4,7,6,8,11,4]
```

Should a pattern match fail, it will just move on to the next element.

Lists themselves can also be used in pattern matching. You can match with the empty list `[]` or any pattern that involves `:` and the empty list. But since `[1,2,3]` is just syntactic sugar for `1:2:3:[]`, you can also use the former pattern. A pattern like `x:xs` will bind the head of the list to `x` and the rest of it to `xs`, even if there's only one element so `xs` ends up being an empty list.

Note: The `x:xs` pattern is used a lot, especially with recursive functions. But patterns that have `:` in them only match against lists of length 1 or more.

If you want to bind, say, the first three elements to variables and the rest of the list to another variable, you can use something like `x:y:z:zs`. It will only match against lists that have three elements or more.

Now that we know how to pattern match against list, let's make our own implementation of the `head` function.

```
head' :: [a] -> a
head' [] = error "Can't call head on an empty list, dummy!"
head' (x:_) = x
```

Checking if it works:

```
ghci> head' [4,5,6]
4
ghci> head' "Hello"
'H'
```

Nice! Notice that if you want to bind to several variables (even if one of them is just `_` and doesn't actually bind at all), we have to surround them in parentheses. Also notice the `error` function that we used. It takes a string and generates a runtime error, using that string as information about what kind of error occurred. It causes the program to crash, so it's not good to use it too much. But calling `head` on an empty list doesn't make sense.

Let's make a trivial function that tells us some of the first elements of the list in (in)convenient English form.

```
tell :: (Show a) => [a] -> String
tell [] = "The list is empty"
tell (x:[]) = "The list has one element: " ++ show x
tell (x:y:[]) = "The list has two elements: " ++ show x ++ " and " ++ show y
tell (x:y:_) = "This list is long. The first two elements are: " ++ show x ++ " and "
```

This function is safe because it takes care of the empty list, a singleton list, a list with two elements and a list with more than two elements. Note that `(x:[])` and `(x:y:[])` could be rewritten as `[x]` and `[x,y]` (because its syntactic sugar, we don't need the parentheses). We can't rewrite `(x:y:_)` with square brackets because it matches any list of length 2 or more.

We already implemented our own `length` function using list comprehension. Now we'll do it by using pattern matching and a little recursion:

```
length' :: (Num b) => [a] -> b
length' [] = 0
length' (_:xs) = 1 + length' xs
```

This is similar to the factorial function we wrote earlier. First we defined the result of a known input — the empty list. This is also known as the edge condition. Then in the second pattern we take the list apart by splitting it into a head and a tail. We say that the length is equal to 1 plus the length of the tail. We use `_` to match the head because we don't actually care what it is. Also note that we've taken care of all possible patterns of a list. The first pattern matches an empty list and the second one matches anything that isn't an empty list.

Let's see what happens if we call `length'` on `"ham"`. First, it will check if it's an empty list. Because it isn't, it falls through to the second pattern. It matches on the second pattern and there it says that the length is `1 + length' "am"`, because we broke it into a head and a tail and discarded the head. Okay. The `length'` of `"am"` is, similarly, `1 + length' "m"`. So right now we have `1 + (1 + length' "m")`. `length' "m"` is `1 + length' ""` (could also be written as `1 + length' []`). And we've defined `length' []` to be `0`. So in the end we have `1 + (1 + (1 + 0))`.

Let's implement `sum`. We know that the sum of an empty list is 0. We write that down as a pattern. And we also know that the sum of a list is the head plus the sum of the rest of the list. So if we write that down, we get:

```
sum' :: (Num a) => [a] -> a
sum' [] = 0
sum' (x:xs) = x + sum' xs
```

There's also a thing called *as patterns*. Those are a handy way of breaking something up according to a pattern and binding it to names whilst still keeping a reference to the whole thing. You do that by putting a name and an `@` in front of a pattern. For instance, the pattern `xs@(x:y:ys)`. This pattern will match exactly the same thing as `x:y:ys` but you can easily get the whole list via `xs` instead of repeating yourself by typing out `x:y:ys` in the function body again. Here's a quick and dirty example:

```
capital :: String -> String
capital "" = "Empty string, whoops!"
```

```
capital all@(x:xs) = "The first letter of " ++ all ++ " is " ++ [x]
```

```
ghci> capital "Dracula"
"The first letter of Dracula is D"
```

Normally we use `as` patterns to avoid repeating ourselves when matching against a bigger pattern when we have to use the whole thing again in the function body.

One more thing — you can't use `++` in pattern matches. If you tried to pattern match against `(xs ++ ys)`, what would be in the first and what would be in the second list? It doesn't make much sense. It would make sense to match stuff against `(xs ++ [x,y,z])` or just `(xs ++ [x])`, but because of the nature of lists, you can't do that.

Guards, guards!



Whereas patterns are a way of making sure a value conforms to some form and deconstructing it, guards are a way of testing whether some property of a value (or several of them) are true or false. That sounds a lot like an if statement and it's very similar. The thing is that guards are a lot more readable when you have several conditions and they play really nicely with patterns.

Instead of explaining their syntax, let's just dive in and make a function using guards.

We're going to make a simple function that responds differently depending on the [density](#) given. Density (or specific mass) is a substance's mass per unit of volume (here, grams per liter). If a substance has a density of less than 1.2, it will float in air, as 1.2g/L is the density of air. If it has more than 1000g/L (the density of water), it will sink in water. Between are things (like people, usually) that will neither float away nor sink in water. So here's the function (we won't be calculating density right now, this function just gets a density and responds)

```
densityTell :: (RealFloat a) => a -> String
densityTell density
  | density < 1.2 = "Wow! You're going for a ride in the sky!"
  | density <= 1000.0 = "Have fun swimming, but watch out for sharks!"
  | otherwise    = "If it's sink or swim, you're going to sink."
```

Guards are indicated by pipes that follow a function's name and its parameters. Usually, they're indented a bit to the right and lined up. A guard is basically a boolean expression. If it evaluates to `True`, then the corresponding function body is used. If it evaluates to `False`, checking drops through to the next guard and so on. If we call this function with `24.3`, it will first check if that's smaller than `1.2`. Because it isn't, it falls through to the next guard. The check is carried out with the second guard and because `24.3` is less than `1000.0`, the second string is returned.

This is very reminiscent of a big if else tree in imperative languages, only this is far better and more readable. While big if else trees are usually frowned upon, sometimes a problem is defined in such a discrete way that you can't get around them. Guards are a very nice alternative for this.

Many times, the last guard is **otherwise**. **otherwise** is defined simply as **otherwise = True** and catches everything. This is very similar to patterns, only they check if the input satisfies a pattern but guards check for boolean conditions. If all the guards of a function evaluate to **False** (and we haven't provided an **otherwise** catch-all guard), evaluation falls through to the next **pattern**. That's how patterns and guards play nicely together. If no suitable guards or patterns are found, an error is thrown.

Of course we can use guards with functions that take as many parameters as we want. Instead of having the user calculate the density of the substance on their own before calling the function, let's modify this function so that it takes a mass (in grams) and volume (in liters).

```
densityTell :: (RealFloat a) => a -> a -> String
densityTell mass volume
  | mass / volume < 1.2 = "Wow! You're going for a ride in the sky!"
  | mass / volume <= 1000.0 = "Have fun swimming, but watch out for sharks!"
  | otherwise = "If it's sink or swim, you're going to sink."
```

Let's see if cat food will float ...

```
ghci> densityTell 400 1
"Have fun swimming, but watch out for sharks!"
```

Looks like it will! At least until it dissolves into the pool... Yuck!

Note that there's no **=** right after the function name and its parameters, before the first guard. Many newbies get syntax errors because they sometimes put it there.

Another very simple example: let's implement our own **max** function. If you remember, it takes two things that can be compared and returns the larger of them.

```
max' :: (Ord a) => a -> a -> a
max' a b
  | a > b = a
  | otherwise = b
```

Guards can also be written inline, although I'd advise against that because it's less readable, even for very short functions. But to demonstrate, we could write **max'** like this:

```
max' :: (Ord a) => a -> a -> a
```

```
max' a b | a > b = a | otherwise = b
```

Ugh! Not very readable at all! Moving on: let's implement our own `compare` by using guards.

```
myCompare :: (Ord a) => a -> a -> Ordering
a `myCompare` b
  | a > b      = GT
  | a == b     = EQ
  | otherwise  = LT
```

```
ghci> 3 `myCompare` 2
GT
```

Note: Not only can we call functions as infix with backticks, we can also define them using backticks. Sometimes it's easier to read that way.

Where!?

In the previous section, we defined a density calculator function and responder like this:

```
densityTell :: (RealFloat a) => a -> a -> String
densityTell mass volume
  | mass / volume < 1.2 = "Wow! You're going for a ride in the sky!"
  | mass / volume <= 1000.0 = "Have fun swimming, but watch out for sharks!"
  | otherwise          = "If it's sink or swim, you're going to sink."
```

Notice that we repeat ourselves here two times. We repeat ourselves two times. Repeating yourself (two times) while programming is about as desirable as getting kicked inna head. Since we repeat the same expression twice, it would be ideal if we could calculate it once, bind it to a name and then use that name instead of the expression. Well, we can modify our function like this:

```
densityTell :: (RealFloat a) => a -> a -> String
densityTell mass volume
  | density < 1.2 = "Wow! You're going for a ride in the sky!"
  | density <= 1000.0 = "Have fun swimming, but watch out for sharks!"
  | otherwise      = "If it's sink or swim, you're going to sink."
  where density = mass / volume
```

We put the keyword `where` after the guards (usually it's best to indent it as much as the pipes are indented) and then we define several names or functions. These names are visible across the guards and give us the advantage of not having to repeat ourselves. If we decide that we want to calculate density a bit differently, we only have to change it once. It also improves readability by giving names to

things and can make our programs faster since stuff like our `density` variable here is calculated only once. We could go a bit overboard and present our function like this:

```
densityTell :: (RealFloat a) => a -> a -> String
densityTell mass volume
  | density < air = "Wow! You're going for a ride in the sky!"
  | density <= water = "Have fun swimming, but watch out for sharks!"
  | otherwise    = "If it's sink or swim, you're going to sink."
  where density = mass / volume
        air = 1.2
        water = 1000.0
```

The names we define in the `where` block of a function are only visible to that function, so we don't have to worry about them polluting the namespace of other functions. Notice that all the names are aligned at a single column. If we don't align them nice and proper, Haskell gets confused because then it doesn't know they're all part of the same block.

`where` bindings aren't shared across function bodies of different patterns. If you want several patterns of one function to access some shared name, you have to define it globally.

You can also use `where` bindings to **pattern match**! We could have rewritten the `where` block of our previous function as:

```
...
  where density = mass / volume
        (air, water) = (1.2, 1000.0)
```

Let's make another fairly trivial function where we get a first and a last name and give someone back their initials.

```
initials :: String -> String -> String
initials firstname lastname = [f] ++ ". " ++ [l] ++ "."
  where (f:_) = firstname
        (l:_) = lastname
```

We could have done this pattern matching directly in the function's parameters (it would have been shorter and clearer actually) but this just goes to show that it's possible to do it in `where` bindings as well.

Just like we've defined constants in `where` blocks, you can also define functions. Staying true to our solids programming theme, let's make a function that takes a list of mass-volume pairs and returns a list of densities.

```
calcDensities :: (RealFloat a) => [(a, a)] -> [a]
calcDensities xs = [density m v | (m, v) <- xs]
  where density mass volume = mass / volume
```

And that's all there is to it! The reason we had to introduce **density** as a function in this example is because we can't just calculate one density from the function's parameters. We have to examine the list passed to the function and there's a different density for every pair in there.

where bindings can also be nested. It's a common idiom to make a function and define some helper function in its **where** block and then to give those functions helper functions as well, each with its own **where** block.

Let it be

Very similar to **where** bindings are **let** bindings. **where** bindings are a syntactic construct that let you bind to variables at the end of a function and the whole function can see them, including all the guards.

let bindings let you bind to variables anywhere and are expressions themselves, but are very local, so they don't span across guards. Just like any construct in Haskell that is used to bind values to names,

let bindings can be used for pattern matching. Let's see them in action! This is how we could define a function that gives us a cylinder's surface area based on its height and radius:

```
cylinder :: (RealFloat a) => a -> a -> a
cylinder r h =
  let sideArea = 2 * pi * r * h
      topArea = pi * r ^2
  in sideArea + 2 * topArea
```

The form is **let <bindings> in <expression>**. The names that you define in the **let** part are accessible to the expression after the **in** part. As you can see, we could have also defined this with a **where** binding. Notice that the names are also aligned in a single column. So what's the difference between the two? For now it just seems that **let** puts the bindings first and the expression that uses them later whereas **where** is the other way around.

The difference is that **let** bindings are expressions themselves. **where** bindings are just syntactic constructs. Remember when we did the if statement and it was explained that an if else statement is an expression and you can cram it in almost anywhere?



```
ghci> [if 5 > 3 then "Woo" else "Boo", if 'a' > 'b' then "Foo" else "Bar"]
["Woo", "Bar"]
ghci> 4 * (if 10 > 5 then 10 else 0) + 2
```

42

You can also do that with `let` bindings.

```
ghci> 4 * (let a = 9 in a + 1) + 2
42
```

They can also be used to introduce functions in a local scope:

```
ghci> [let square x = x * x in (square 5, square 3, square 2)]
[(25,9,4)]
```

If we want to bind to several variables inline, we obviously can't align them at columns. That's why we can separate them with semicolons.

```
ghci> (let a = 100; b = 200; c = 300 in a*b*c, let foo="Hey "; bar = "there!" in fo
(6000000,"Hey there!")
```



You don't have to put a semicolon after the last binding but you can if you want. Like we said before, you can pattern match with `let` bindings. They're very useful for quickly dismantling a tuple into components and binding them to names and such.

```
ghci> (let (a,b,c) = (1,2,3) in a+b+c) * 100
600
```

You can also put `let` bindings inside list comprehensions. Let's rewrite our previous example of calculating lists of mass-volume pairs to use a `let` inside a list comprehension instead of defining an auxiliary function with a `where`.

```
calcDensities :: (RealFloat a) => [(a, a)] -> [a]
calcDensities xs = [density | (m, v) <- xs, let density = m / v]
```

We include a `let` inside a list comprehension much like we would a predicate, only it doesn't filter the list, it only binds to names. The names defined in a `let` inside a list comprehension are visible to the output function (the part before the `|`) and all predicates and sections that come after of the binding. So we could make our function return only the densities that will float in air:

```
calcDensities :: (RealFloat a) => [(a, a)] -> [a]
```

```
calcDensities xs = [density | (m, v) <-
  xs, let density = m / v, density < 1.2]
```

We can't use the `density` name in the `(m, v) <- xs` part because it's defined prior to the `let` binding.

We omitted the `in` part of the `let` binding when we used them in list comprehensions because the visibility of the names is already predefined there. However, we could use a `let in` binding in a predicate and the names defined would only be visible to that predicate. The `in` part can also be omitted when defining functions and constants directly in GHCi. If we do that, then the names will be visible throughout the entire interactive session.

```
ghci> let zoot x y z = x * y + z
ghci> zoot 3 9 2
29
ghci> let boot x y z = x * y + z in boot 3 4 2
14
ghci> boot
<interactive>:1:0: Not in scope: `boot`
```

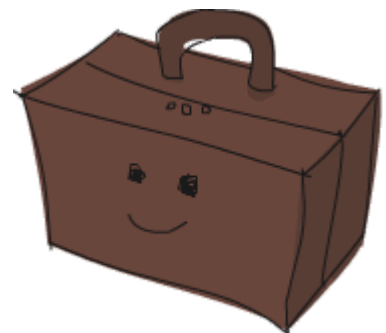
Let's not.

Back in the day, using `let` used to be required when binding in GHCi. A bit inconvenient! These days you can omit the `let` and enjoy exactly the same result.

If `let` bindings are so cool, why not use them all the time instead of `where` bindings, you ask? Well, since `let` bindings are expressions and are fairly local in their scope, they can't be used across guards. Some people prefer `where` bindings because the names come after the function they're being used in. That way, the function body is closer to its name and type declaration and to some that's more readable.

Case expressions

Many imperative languages (C, C++, Java, etc.) have case syntax and if you've ever programmed in them, you probably know what it's about. It's about taking a variable and then executing blocks of code for specific values of that variable and then maybe including a catch-all block of code in case the variable has some value for which we didn't set up a case.



Haskell takes that concept and one-ups it. Like the name implies, case expressions are, well, expressions, much like if else expressions and `let` bindings. Not only can we evaluate expressions based on the possible cases of the value of a variable, we can also do pattern matching. Hmmm, taking

a variable, pattern matching it, evaluating pieces of code based on its value, where have we heard this before? Oh yeah, pattern matching on parameters in function definitions! Well, that's actually just syntactic sugar for case expressions. These two pieces of code do the same thing and are interchangeable:

```
head' :: [a] -> a
head' [] = error "No head for empty lists!"
head' (x:_) = x

head' :: [a] -> a
head' xs = case xs of [] -> error "No head for empty lists!"
                  (x:_) -> x
```

As you can see, the syntax for case expressions is pretty simple:

```
case expression of pattern -> result
                  pattern -> result
                  pattern -> result
                  ...
```

expression is matched against the patterns. The pattern matching action is the same as expected: the first pattern that matches the expression is used. If it falls through the whole case expression and no suitable pattern is found, a runtime error occurs.

Whereas pattern matching on function parameters can only be done when defining functions, case expressions can be used pretty much anywhere. For instance:

```
describeList :: [a] -> String
describeList xs = "The list is " ++ case xs of [] -> "empty."
                                             [x] -> "a singleton list."
                                             xs -> "a longer list."
```

They are useful for pattern matching against something in the middle of an expression. Because pattern matching in function definitions is syntactic sugar for case expressions, we could have also defined this like so:

```
describeList :: [a] -> String
describeList xs = "The list is " ++ what xs
    where what [] = "empty."
          what [x] = "a singleton list."
          what xs = "a longer list."
```

